Exact Rational Integration over Constrained Simplices

Symmetric Polynomial Bases, Marginal Geometry, and Lean 4 Verification

Reading Point (translatepi.ai)

Draft v1.1 — 23 October 2026

Abstract

We present formally verified integration identities for polynomial weights on simplexes constrained by coordinate thresholds and cardinality-dependent caps. The development establishes exact simplex moments, a rational polynomial subspace of dimension 846, general monomial and ordinary Gram-entry formulas, fibre decompositions, two-aggregate Dirichlet integration, and exact polygon-cell integrals covering all marginal endpoint branches. Kernel-checked rational certificates first establish the marginal entries $J_T[1,1]$, $J_T[P_2,1]$, and $J_T[P_2,P_2]$, completing a 2×2 block. The V2i development then proves a general exact rational representation for every pair of polynomial generators, with further Table 3 certificates and an exact bridge back to the V2h entry. The V2i 33-statement Comparator suite was accepted by the Lean default kernel on the VPS; its full regression produced 21 of 21 acceptances and exited with status zero. V2j adds a formally checked shared-moment aggregation layer: repeated moment evaluations are grouped by keys and weighted sums of Gram entries are proved equal to their unaggregated V2i expressions. Its 24-statement Comparator configuration was accepted on both a macOS development installation and a Linux VPS. V2k replaces the coordinate-placement recursion with a canonical symmetric-function evaluator; its equality with V2i is formally proved for every moment oracle, class-size vector, and factor list. The 36-statement V2k Comparator suite was accepted on the MacBook Pro and Linux VPS. V2ℓ proves staged step and chain certificates, factor-order invariance, and reusable moment profiles. The 35-statement Comparator suite was independently accepted on the Linux VPS at proof commit `8033c48`. Staged degree-twelve demonstrations represent one Table 3 piece using 231 moment keys, without asserting evaluation of all moment values. V2m extends this to a certified prefix trie with 77 nonroot nodes and 78 moment profiles, including degree-forty coefficient-table examples. Its sequential certificate build succeeded on macOS and Linux; the 20-statement Comparator was accepted by the Lean default kernel on the Linux VPS. These results give exact representations and selected certificates, not an evaluated 846×846 numerical matrix or the final variational inequality.

1 Introduction and scope

Exact integration on constrained regions is useful where a variational inequality must ultimately be checked using rational arithmetic. A conventional simplex has a closed monomial moment formula; the present support also depends on which coordinates cross a threshold and on a cap for the sum of those coordinates. This introduces a finite stratification, but preserves a route to exact, rational expressions.

We study the mathematical constructions in the public Lean repository:

`github.com/thinkthoughts/lean-bgp212`

The motivating bounded-prime-gap argument originates with Axiom Math [1]; the statements below are an independent formalization of specified components and do not amount to a complete formal proof of the unconditional prime-gap bound. References to Lean declarations are included to delimit precisely what has been checked. The short arguments given here are *proof outlines*, not substitutes for the complete Lean proof terms.

2 Polynomial moments on a simplex

Definition 2.1. *For $m \in \mathbb{N}$ and $R \in \mathbb{R}$, define*

$$\Delta_m(R) = \left\{ x \in \mathbb{R}^m : x_i \geq 0 \text{ for all } i, \quad \sum_{i=1}^{m} x_i \leq R \right\}.$$

Theorem 2.2 (Simplex monomial moments)**.** *For $m \in \mathbb{N}$, $R \geq 0$, and $e \in \mathbb{N}^m$,*

$$\int_{\Delta_m(R)} \prod_{i=1}^{m} x_i^{e_i}\, dx = \frac{R^{m+\sum_i e_i} \prod_i (e_i!)}{(m + \sum_i e_i)!}.$$

Here the dimension-zero case uses the corresponding Lean integral convention.

Lean: `simplex_integral`; definitions `simplex`, `momentR`.
Source: `Solution/SimplexMoment.lean:160`.

Proof outline. Induct on dimension and use Fubini to isolate a coordinate. The inner integral is a Beta integral; the factorial expression then simplifies by induction. The formal development additionally treats negative and zero radii, simplex volume, and rational-polynomial integrals (`poly_integral`).

3 An indexed rational polynomial space

Consider rational polynomials in 45 coordinates. Write P_j for the jth power sum, P_λ for the product over parts of a multiset λ, and set

$$G_{a,\lambda} = (L - P_1)^a P_\lambda, \qquad \lambda \text{ has positive even parts}, \quad a + |\lambda| \leq 21.$$

The index type $B_{21}\mathrm{Idx}$ enumerates the admissible pairs.

Theorem 3.1 (Independence and rank)**.** *The indexed family $G_{a,\lambda}$ is linearly independent over $\mathbb{Q}$. Its rational span B_{21} has*

$$\dim_{\mathbb{Q}} B_{21} = 846.$$

Lean: `genIdx_linearIndependent`, `finrank_B21` (`Solution/Variational.lean:398,410`).

Proof outline. The indexing is injective; distinct indexed monomials are linearly independent. Injective polynomial transformations preserve this independence. A separate count establishes $|B_{21}\mathrm{Idx}| = 846$, and the span-rank identity gives the result. The theorem concerns this designated subspace, not the full polynomial ring.

4 Cap-constrained supports and general integrals

Let $N \in \mathbb{N}$, $g \geq 0$, $L \in \mathbb{R}$, and $C : \mathbb{N} \to \mathbb{R}$. For $x \in \mathbb{R}^N$ write

$$R_g(x) = \{i : x_i > g\}, \qquad s_g(x) = |R_g(x)|, \qquad \rho_g(x) = \sum_{i \in R_g(x)} x_i.$$

Define the capped simplex support

$$T(N, g, L, C) = \left\{ x \in \mathbb{R}^N : x_i \geq 0, \quad \sum_i x_i \leq L, \quad \rho_g(x) \leq C(s_g(x)) \right\}. \tag{1}$$

Theorem 4.1 (General monomial integral)**.** *For $N \in \mathbb{N}$, $g \geq 0$, $L \in \mathbb{R}$, $C : \mathbb{N} \to \mathbb{R}$, $e \in \mathbb{N}^N$, and $p \in \mathbb{N}$,*

$$\int_{T(N,g,L,C)} \left(L - \sum_i x_i \right)^p \prod_i x_i^{e_i} \, dx = \mathrm{gramGen}(N, g, L, C, e, p). \tag{2}$$

The right-hand side is the explicit, finite expression defined in Lean.

Lean: `integral_supportT_gen`; `gramGen`.
Sources: `Solution/GramGeneral.lean:344`;
`BGP212/GramGeneral.lean`.

Proof outline. Stratify by the rough-coordinate set, apply inclusion–exclusion to threshold conditions, and translate the resulting pieces to weighted simplexes. Use Theorem 2.2 and Fubini to evaluate the polynomial weights. Placement-counting identities `sig_insert`, `sig_union`, and `sig_card` reduce the sums to the finite closed form. The reduction `gramGen_extend` transports calculation to the coordinates having nonzero exponents. This is an algebraic efficiency result, not a claim that all entries are computationally cheap to certify.

5 Rational Gram representations

Specialize to the fixed rational cap values and support parameters of Table 3 in [1], with $N = 45$. For indices $i, j \in B_{21}\mathrm{Idx}$ define

$$I_T(i, j) = \int_T G_i(x) G_j(x) \, dx.$$

Theorem 5.1 (Explicit rational representation)**.** *For every $i, j \in B_{21}\mathrm{Idx}$, $I_T(i, j)$ equals the real embedding of a finite rational sum indexed by placements φ of the combined power-sum parts:*

$$I_T(i, j) = \left(\sum_{\varphi} \mathrm{gramGen}_{\mathbb{Q}}(45, g, L, C, e_\varphi, a_i + a_j) \right)_{\mathbb{R}}.$$

Here e_φ is the exponent vector obtained from the placement, exactly as specified by `expo` *in the Lean source.*

Lean: `gram_entry_B21_rat` (`Solution/GramGeneral.lean:522`); `gram_entry_gen_list` and `gram_entry_example`.

The theorem gives each entry an explicit rational representation; it does not supply an evaluated and certified 846×846 matrix. One worked entry, corresponding to $(L - P_1)P_2^2$, has its value certified using a collection of small exact rational lemmas. This separation between an algebraic identity and its numerical certificate is important to reproducibility.

6 Marginal-fibre geometry

Let $\widehat{x} \in \mathbb{R}^n$ and append $u \in \mathbb{R}$ as the last coordinate. Define $S = \sum_i \widehat{x}_i$, $s = s_g(\widehat{x})$, $\rho_1 = \rho_g(\widehat{x})$, and

$$\widehat{T} = \{\widehat{x} \geq 0 : S \leq \tau, \ \rho_1 \leq C(s)\}, \quad U(\widehat{x}) = \min\{L - S, \max(g, C(s+1) - \rho_1)\}.$$

Theorem 6.1 (Fibre equivalence)**.** *Suppose $g \geq 0$, $\tau \leq L$, and $\widehat{x} \in \widehat{T}$. Then for every $u \in \mathbb{R}$,*

$$(\widehat{x}, u) \in T(n+1, g, L, C) \quad \Longleftrightarrow \quad 0 \leq u \leq U(\widehat{x}).$$

Lean: `fibre_equiv` (`Solution/Fibre.lean:115`).

Proof outline. Appending u leaves the rough set unchanged for $u \le g$, and increments both rough count and rough mass for $u > g$. Combining the resulting cap inequality with $S + u \le L$ identifies the minimum–maximum endpoint exactly. The empty-fibre theorem for cap-violating outer coordinates additionally needs $C(m+1) \le C(m) + g$ for $m \ge 1$, and $C(0) \ge 0$. No constraint on the step $m = 0$ is assumed; the relevant Table 3 facts are checked in Lean.

Theorem 6.2 (Marginal identity)**.** *Assume $g \ge 0$, $\tau \le L$, $C(0) \ge 0$, and $C(m+1) \le C(m) + g$ for every $m \ge 1$. For continuous P,*

$$\mathrm{marginalJ}(n, g, L, \tau, C, P) = J_T(n, g, L, \tau, C, P).$$

The right-hand functional integrates the squared fibre integral over $\widehat{T}$.

Lean: `marginal_agrees` (`Solution/Fibre.lean:217`).

Proposition 6.3 (Polynomial inner integral)**.** *For arbitrary $U, M \in \mathbb{R}$ and $p, t \in \mathbb{N}$,*

$$\int_0^U (M-u)^p u^t \, du = \sum_{j=0}^{p} \binom{p}{j} M^{p-j} (-1)^j \frac{U^{t+j+1}}{t+j+1}.$$

Lean: `inner_integral` (`Solution/Fibre.lean:240`). The equality follows by expanding the binomial and integrating each power. It holds for arbitrary real U under the oriented interval-integral convention.

7 Two-aggregate Dirichlet integration

Let I and J be disjoint, nonempty finite coordinate groups, with exponent vectors $a \in \mathbb{N}^I$ and $b \in \mathbb{N}^J$. For a nonempty finite group G and an exponent vector e, define

$$\alpha_G(e) = |G| + \sum_{i \in G} e_i - 1, \qquad A_G(e) = \frac{\prod_{i \in G}(e_i!)}{\alpha_G(e)!}.$$

Write $\Delta_{I \sqcup J}(M)$ for the nonnegative simplex with total coordinate sum at most M, and

$$\Delta_2(M) = \{(u_1, u_2) \in \mathbb{R}^2_{\ge 0} : u_1 + u_2 \le M\}.$$

Theorem 7.1 (Two-aggregate Dirichlet integration)**.** *Suppose I and J are nonempty, and $h : \mathbb{R}^2 \to \mathbb{R}$ is measurable and bounded on $\Delta_2(M)$. With $u_1 = \sum_{i \in I} y_i$ and $u_2 = \sum_{j \in J} y_j$, the weighted simplex integral satisfies*

$$\begin{aligned} &\int_{\Delta_{I \sqcup J}(M)} \left(\prod_{i \in I} y_i^{a_i} \right) \left(\prod_{j \in J} y_j^{b_j} \right) h(u_1, u_2) \, dy \\ &\qquad = A_I(a) A_J(b) \int_{\Delta_2(M)} u_1^{\alpha_I(a)} u_2^{\alpha_J(b)} h(u_1, u_2) \, du_1 \, du_2. \end{aligned} \tag{3}$$

The formalization also gives the version on $\mathrm{Fin}\, N$ *for a rough coordinate set and its complement.*

Lean: `two_group`, `two_group_fin`, `one_group`; `Solution/Aggregate.lean`. The theorem is proved for the corresponding measurable, bounded integrands; separate theorems handle empty groups and other boundary cases.

Proof outline. Use Fubini and a sum-coordinate shear to isolate the total in one group; apply the weighted simplex moment calculation to the corresponding slices. Repeat for the second group. The factors A_I, A_J record the Dirichlet normalizations; in particular the -1 in each aggregate exponent is essential. The polynomial cases `tri_poly` and `two_group_poly` yield explicit rational expressions, and `two_group_const` recovers the simplex moment formula when $h = 1$.

Proposition 7.2 (A cut-triangle certificate)**.** *With $y = (y_0, y_1, y_2)$, the exact value is*

$$\int_{\substack{y_i \geq 0,\ y_0+y_1+y_2 \leq 1 \\ y_0 \leq 1/2}} y_0 y_2^2 \, dy = \frac{19}{7680}.$$

Lean: `tri_cut`, `example_value` (`Solution/Aggregate.lean`). The certificate reduces this integral to $\frac{1}{3}\int_{\Delta_2(1),\, u_1 \leq 1/2} u_1 u_2^3 \, du_1 du_2$; the value has also been cross-checked by direct rational integration.

Connection to marginal geometry. On the translated stratum $x = y + g\mathbf{1}_{R\cup K}$, define $u_1 = \sum_{i\in R} y_i$ and $u_2 = \sum_{i\notin R} y_i$. The proved correspondence `stratumEndpoint_shift` expresses the fibre endpoint as

$$U = \min\big(L - (r+k)g - u_1 - u_2,\ \max\big(g, C(r+1) - rg - u_1\big)\big), \qquad r = |R|,\ k = |K|.$$

The cap index belongs to the stratum R. Substituting the translated point's rough set $R \cup K$ instead changes the endpoint; the Table 3 control records the distinct values 587/1250 and 461/1250. The cap-branch polygon-cell integration is established in Section 8; all endpoint branches and the constant marginal Gram entry are treated in Section 9; the power-sum entry follows in Section 10, and the power-sum-square entry in Section 11, and general generator entries in Section 12.

8 Polygon-cell integration and the cap branch

The two-aggregate reduction produces planar regions bounded by coordinate thresholds and a total-sum constraint. For real A, B, D define the clipped cell

$$\mathcal{B}(A, B, D) = \{(u_1, u_2) \in \mathbb{R}^2 : 0 \leq u_1 \leq A,\ 0 \leq u_2 \leq B,\ u_1 + u_2 \leq D\}.$$

Theorem 8.1 (Exact polygon-cell moments)**.** *For $\alpha, \beta \in \mathbb{N}$, the Lebesgue integral*

$$\int_{\mathcal{B}(A,B,D)} u_1^\alpha u_2^\beta \, du_1 \, du_2 = \mathrm{boxVal}(A, B, D, \alpha, \beta)$$

where boxVal *is a finite closed form using polynomial antiderivatives. For rational A, B, D its value is rational. The same construction gives rational integrals for arbitrary rational polynomials in two variables. Degenerate and empty cells are treated separately.*

Lean: `box_monomial`, `boxVal_cast`, `box_poly`, `boxVal_empty`, `boxVal_nonpos`, `boxVal_rect`; `Solution/CapCell.lean`.

Proof outline. Integrate first in u_2 with upper endpoint $\min(B, D - u_1)$ and split the u_1 interval at $D - B$. The resulting one-variable powers have exact polynomial antiderivatives. In particular, when $A + B \leq D$ the diagonal constraint is redundant and the cell is a rectangle; the formula also covers diagonally truncated cells.

Theorem 8.2 (Cap-branch endpoint and piece integral)**.** *On a translated stratum $x = y + g\mathbf{1}_{R\cup K}$, with $r = |R|$, $k = |K|$ and aggregates u_1, u_2 as in Section 7, the cap branch of the fibre endpoint is*

$$U = C(r+1) - rg - u_1,$$

subject to its vertical and horizontal branch constraints. Combined with the piece constraints, the aggregate domain is a clipped cell $\mathcal{B}(A, B, D)$ with parameters determined by g, L, τ, C, r, k. The weighted piece integrals

$$\int_{\text{piece}} y^t \, \mathbf{1}_{\text{cap}} U^m \, dy$$

therefore equal an explicit expression obtained from the two-aggregate Dirichlet constants and rational cell integral formulas.

Lean: `endpointAgg_on_capCell`, `cap_branch_piece`, `cap_branch_r0`, `cap_inner`; `Solution/CapCell.lean`. The formalization proves the branch classification and corresponding endpoint equalities. For Table 3 the τ constraint is redundant on the cap branch: $\tau - (L - g) = 1/625 > 0$, and the cap cells become rectangles (`tau_ge_table3`, `capCell_rect_table3`).

Proposition 8.3 (A Table 3 cap-piece certificate)**.** *For the specified 44-coordinate piece $(R, K) = (\{0\}, \varnothing)$, the cap-branch contribution to the integral defining $J_T(1)$ is*

$$\frac{(L - C(2))^{43}\big((C(2) - g)^3 - g^3\big)}{3 \cdot 43!} \approx 1.0464 \times 10^{-70}.$$

Lean: `capPieceT3_value` (`Solution/CapCell.lean`). This is *one piece* of the inclusion–exclusion decomposition, not the complete value of $J_T(1)$. The exact rational certificate is kernel-checked. Polygon clipping and lower-dimensional integration provide independent numerical controls; the latter is an approximation, not part of the kernel proof. The remaining branches and their assembly are treated in Section 9.

9 All endpoint branches and the first marginal Gram entry

The translated marginal endpoint is the minimum of the remaining total mass and the maximum of a cap expression and g:

$$U = \min\big(L - (r+k)g - u_1 - u_2,\ \max(g, C(r+1) - rg - u_1)\big).$$

Its switching lines partition each two-aggregate piece into four cells: the cap branch, the remaining-mass branch on the cap side, the constant g branch, and the remaining-mass branch on the g side. Each cell is represented by signed differences of the clipped polygon cells $\mathcal{B}(A, B, D)$ in Section 8.

Theorem 9.1 (Exact branch-cell and piece integrals)**.** *For nonnegative integer exponents, the weighted integral on each branch cell of a polynomial block*

$$u_1^{\alpha} u_2^{\beta} (p - u_1)^{m_1} (L' - u_1 - u_2)^{m_2}$$

reduces to a finite combination of exact clipped-cell moments. The corresponding integrals on translated rough-set strata admit closed forms, including the stratum with no rough coordinates. For integrands depending on the stratum, inclusion–exclusion and cardinality counting assemble these pieces into the marginal functional.

Lean: `box_twoPow`, `cells_integral`, `piece_block`, `piece_r0`, `indicator_supportT_strat`, `count_by_card`; `Solution/MarginalEntry.lean`.

Theorem 9.2 (Rational reduction of the constant-generator marginal)**.** *For $N \geq 1$ and parameters satisfying $0 \leq g \leq C(1)$ and $C(N) < Ng$, the squared-fibre marginal for the constant polynomial is expressed by the explicit rational-piece expression* jtSum*:*

$$J_T(1) = (N+1) \int_{\widehat{T}} U(\widehat{x})^2 \, d\widehat{x} = \mathrm{jtSum}(N, g, L, \tau, C, 2, 0).$$

Lean: `jT_one_eq`, `jT_one_pieces`, `piece_eq_pieceJ`; `Solution/MarginalEntry.lean`. The reduction uses binomial counting for this symmetric integrand; entries with power-sum factors require additional expansion and placement accounting.

Proposition 9.3 (Table 3 marginal Gram certificate)**.** *Under the Table 3 parameters, for the generator $G_{0,\varnothing} = 1$,*

$$J_T[G_{0,\varnothing}, G_{0,\varnothing}] = 45 \int_{\widehat{T}} U(\widehat{x})^2 \, d\widehat{x} \approx 1.1193487646397893 \times 10^{-55}.$$

The exact value is the ratio of these integers, obtained by concatenating each integer's digit lines (no separators):

Numerator

196405301342598493830210603911544843582810204
587055080769623409436245894353192939488221872
912132381844932553462651053065161718578698716
871038141

Denominator

175463901463993193504011955433551437477604239
482178113383469748717206470461915657210738572
757691144943237304687500000000000000000000000
000000000000000000000000000000000000000000000
0000000000000

Lean: `jtSum_table3`, `jT_one_table3`, `eval_gen_zero`, `jT_gram_gen00_table3`; `Solution/MarginalEntryCert.lean`, `Solution/MarginalEntry.lean`. The proof evaluates 133 nonempty (r, k) pieces by kernel checking and proves the remaining pieces vanish. The exact certificate is regenerated byte-for-byte by `marginal_entry_v2f_emit.py`. Independent polygon clipping, low-dimensional integration, and Monte Carlo provide additional checks, but are not substitutes for Lean verification. This establishes the constant-generator entry. The nonconstant power-sum entry is developed in Section 10.

10 Bilinear marginal form and a power-sum Gram entry (V2g)

Write $P_2(x) = \sum_i x_i^2$, and let $\widehat{P}_2$ denote the same power sum on the first 44 coordinates. The bilinear marginal functional $J_T[P, Q]$ polarizes the squared-fibre integral. Its symmetric bilinearity is formalized by `jTbil_self` and `jTbil_comm`.

Proposition 10.1 (Fibre polynomial for P_2)**.** *For a fibre of length U above $\widehat{x}$, the constant and quadratic polynomial fibre integrals satisfy*

$$\int_0^U 1 \, du = U, \qquad \int_0^U P_2(\widehat{x}, u) \, du = U \widehat{P}_2(\widehat{x}) + \frac{U^3}{3}.$$

Consequently the cross-entry is

$$J_T[P_2, 1] = 45 \int_{\widehat{T}} \left(U^2 \widehat{P}_2 + \frac{U^4}{3} \right) d\widehat{x}. \tag{4}$$

Lean: `inner_P2`, `inner_one`, `p2Entry_eq`; `Solution/PowerSumEntry.lean`.

Coordinate placements. On a translated stratum $x = y + g\mathbf{1}_{R\cup K}$, the power sum becomes

$$\widehat{P}_2(x) = \sum_i y_i^2 + 2g \sum_{i\in R\cup K} y_i + g^2|R \cup K|.$$

One distinguished coordinate lies in R, in K, or in neither; these cases carry multiplicities r, k, and $N - r - k$. The resulting exponent shifts feed the exact two-aggregate Dirichlet weights and polygon-cell formulas established earlier. Theorems `piece_single`, `piece_single_eq`, `sum_place`, and `piece_P2` establish the piecewise and cardinality-weighted assembly, including the necessary boundary cases.

Proposition 10.2 (Table 3 nonconstant marginal certificate)**.** *Under the Table 3 parameters, the marginal Gram cross-entry has an exact rational value $q_{P_2,1} \in \mathbb{Q}$, certified in Lean, with*

$$J_T[P_2, 1] = q_{P_2,1} \approx 4.999439687639965 \times 10^{-57}.$$

The complete numerator and denominator are preserved by the Lean certificate and the reproducible certificate generator.

Lean: `p2EntryVal_table3`, `jT_P2_one_table3`, `jT_one_P2_table3`.
Sources: `Solution/PowerSumEntry.lean`, `Solution/PowerSumEntryCert.lean`.

The exact Table 3 evaluation uses 133 cardinality-class pieces with two certificate lemmas per piece (266 lemmas total); the remaining classes are eliminated by `cond_tail` and `cond_high`. The generator `specs/checks/power_sum_entry_v2g_emit.py` reproduces the Lean certificate byte-for-byte on the VPS. Six independent Lean-file controls have their expected outcomes: two accepted positive controls and four rejected negative controls. Numerical cross-checks include a low-dimensional grid and Monte Carlo; these support the arithmetic independently of the exact kernel proof. The power-sum-square diagonal entry $J_T[P_2, P_2]$, including two distinguished-coordinate placements and their diagonal case, is proved in Section 11.

11 A two-coordinate moment table and the power-sum-square entry (V2h)

The marginal square of the nonconstant generator is

$$J_T[P_2, P_2] = 45 \int_{\widehat{T}} \left(U\widehat{P}_2 + \frac{U^3}{3} \right)^2 d\widehat{x} \tag{5}$$

$$= 45 \int_{\widehat{T}} \left(U^2\widehat{P}_2^2 + \frac{2}{3}U^4\widehat{P}_2 + \frac{1}{9}U^6 \right) d\widehat{x}. \tag{6}$$

The new term contains two distinguished coordinate indices:

$$\widehat{P}_2^2 = \sum_{i,j} x_i^2 x_j^2.$$

For $i = j$ a translated summand becomes $(y_i + s_i)^4$; for $i \neq j$ we obtain $(y_i + s_i)^2(y_j + s_j)^2$, where s_i is the stratum translation (g for coordinates in $R \cup K$, zero otherwise). The three placement classes R, K, and their complement have sizes n_c; the number of ordered distinct-coordinate pairs in classes c, d is $n_c n_d - \mathbf{1}_{c=d} n_c$. The diagonal contributes n_c placements per class.

Theorem 11.1 (Moment-table reduction for the marginal square)**.** *For each translated stratum and each exponent vector t, the weighted piece integral of $y^t U^m$ reduces to a factorial factor $\prod_i t_i!$ multiplied by an exact moment determined by the total monomial degrees in the rough and smooth coordinate groups. Together with the diagonal and distinct-coordinate placement formulas, this gives an exact finite rational representation of (5).*

Lean development: `BGP212/PowerSumPlace.lean`, `BGP212/PowerSumSquare.lean`, `Solution/PowerSumSquare.lean`; `Challenge/PowerSumSquare.lean`. The proof reuses the two-aggregate and polygon-cell machinery, with moments shared among coordinate placements rather than recomputed for each summand.

Proposition 11.2 (Table 3 power-sum-square certificate)**.** *At the specified Table 3 parameters, the entry $J_T[P_2, P_2]$ is an exact rational number represented by the V2h Lean certificate, with*

$$J_T[P_2, P_2] \approx 2.27107 \times 10^{-58}.$$

The certificate is organized into 14 resumable row files and a final assembly file, each checked as part of the V2h Comparator proof.

Certificate sources: `Solution/PowerSumSquareCert0.lean` through `Solution/PowerSumSquareCert13.lean`, `Solution/PowerSumSquareCertSum.lean`; `specs/checks/power_sq_entry_v2h_emit.py`. The 27-statement Comparator configuration is `Comparator/power_sum_square_v2h.json`. Its final VPS output states that the Lean default kernel accepts the solution and reports "Your solution is okay!".

The first 2×2 marginal Gram block. Combining V2f, V2g, and V2h yields the exact rational entries of

$$\mathcal{G}_{\{1,P_2\}} = \begin{pmatrix} J_T[1,1] & J_T[1,P_2] \\ J_T[P_2,1] & J_T[P_2,P_2] \end{pmatrix} \approx \begin{pmatrix} 1.11935 \times 10^{-55} & 4.99944 \times 10^{-57} \\ 4.99944 \times 10^{-57} & 2.27107 \times 10^{-58} \end{pmatrix}.$$

Using the displayed approximations, the Cauchy–Schwarz ratio

$$\frac{J_T[P_2,1]^2}{J_T[1,1]\, J_T[P_2,P_2]} \approx 0.984$$

is close to one. This numerical observation motivates exact assessment of determinants and variational certificates; an independent Lean certificate for a strictly positive 2×2 determinant is not asserted here. Independent Monte Carlo and low-dimensional grid comparisons test the computed value but are distinct from the Lean proof.

12 General polynomial marginal Gram entries (V2i)

The preceding certificates evaluate a small marginal Gram block. V2i extends the exact representation to arbitrary pairs of the generators $G_{a,\lambda} = (L - P_1)^a P_\lambda$, where P_λ is a product of power sums. If $M = L - \sum_i \widehat{x}_i$ and the fibre endpoint is U, then

$$\int_0^U G_{a,\lambda}(\widehat{x}, u)\, du = \sum_{T \subseteq \lambda} \widehat{P}_{\lambda \setminus T} \sum_{j=0}^{a} \binom{a}{j} (-1)^j \frac{M^{a-j} U^{s_T+j+1}}{s_T + j + 1}, \qquad s_T = \sum_{q \in T} q. \tag{7}$$

Here the subset sum counts selections of indexed factors (including repeated parts with their multiplicities). The formula is proved as `inner_genFun` in `Solution/GenEntry.lean`.

Translated placements and common moments. On each stratum, a factor of $\widehat{P}_q$ is a sum of powers of translated coordinates. The recursive placement calculation either attaches a new factor to an already distinguished coordinate or opens a coordinate in one of the three classes (rough, intermediate, or remaining). The latter has multiplicity equal to the class population minus the number of already selected coordinates in that class. Unrolling the recursion accounts for set

partitions and falling-factorial multiplicities. The resulting monomial weights reduce to a factorial factor times a moment indexed by the total rough-group and smooth-group degrees. The principal corresponding Lean declarations include `piece_place`, `piece_state`, and `piece_mono_n`.

Theorem 12.1 (General marginal Gram representation)**.** *For every $N \geq 1$, parameters satisfying the admissibility assumptions $0 \leq g \leq C(1)$ and $C(N) < Ng$, and arbitrary generator indices (a_1, λ_1) and (a_2, λ_2), the bilinear marginal form is an explicit finite expression:*

$$J_T[G_{a_1,\lambda_1}, G_{a_2,\lambda_2}] = \mathrm{genEntryVal}(N, g, L, \tau, C, a_1, \lambda_1, a_2, \lambda_2).$$

For the rational Table 3 parameters, the expression is rational; every pair of generators indexed by B_{21} therefore has an exact rational representation.

Lean: `genEntry_eq`, `jT_gen_table3`; `Solution/GenEntry.lean`, `Solution/GenEntryTable.lean`. The latter theorem supplies a rational representation for all entries; it does not assert that every entry of the numerical matrix has been evaluated.

Proposition 12.2 (Selected Table 3 marginal certificates)**.** *Under the specified Table 3 parameters the exact kernel-checked rational values have the following approximations:*

$$\begin{aligned} J_T[P_2, P_2] &\approx 2.27107 \times 10^{-58}, \\ J_T[P_4, 1] &\approx 2.75169 \times 10^{-59}, \\ J_T[P_{(2,2)}, 1] &\approx 2.28799 \times 10^{-58}, \\ J_T[L - P_1, 1] &\approx 5.52809 \times 10^{-57}. \end{aligned}$$

The first value is recovered exactly from the independently checked V2h certificate by `genEntryPP_eq_v2h`; the other displayed values are further applications of the general formula.

Lean: `jT_P2_P2_gen_table3`, `jT_P4_one_table3`, `jT_P22_one_table3`, `jT_LP1_one_table3`. The supporting rational moment certificates are compiled in `Solution/GenEntryCert0.lean` through `Solution/GenEntryCert13.lean` and `Solution/GenEntryCertSum.lean`. The generator and controls are in `specs/checks/gen_entry_v2i_emit.py` and related `specs/checks/gen_entry_v2i_*.py` files.

Computational scope. The V2i formula covers every pair of generators, whereas compiling an entire numerical Gram matrix would require many further exact evaluations. The optimized polynomial $P^\star$, the exact sign of $J_T(P^\star) - 4I_T(P^\star)$, and the remaining variational conditions are not established here. V2j implements the proposed coefficient aggregation across repeated rational moments, with exact equality to the V2i representation as described in Section 13. The next computational targets include more efficient moment/cell evaluation and additional matrix entries. V2k replaces the placement evaluator by a symmetric-function representation, with an exact identity back to V2i (Section 14).

13 Shared-moment aggregation of marginal Gram entries (V2j)

V2i gives exact formulas for arbitrary marginal Gram entries, but their coordinate-placement expansions can request the same rational moment many times. V2j separates the *moment oracle* from the coefficients of those requests, making it possible to collect repeated keys before evaluation.

Linearity and locality. For fixed geometric data and generator indices, the V2j evaluation `genPieceWith` is linear in its moment oracle. The formal development also constructs a finite, canonically enumerated collection of keys used by the evaluation. If two oracles agree on those keys, the resulting piece values agree. These statements justify replacing repeated oracle calls by a shared lookup table without changing the exact piece value.

Theorem 13.1 (Exact shared-moment aggregation)**.** *For the V2j coefficient aggregation of a specified finite weighted family of generator pairs, the aggregated rational value equals the weighted sum of the corresponding unaggregated V2i entry values. Thus collecting coefficients of identical moment keys preserves the exact value of the weighted marginal Gram expression.*

Lean: `aggEntryVal_eq`, `jT_weighted_eq_agg`, with supporting piece-level linearity, key-locality, and aggregation lemmas; `Solution/MomentAgg.lean` and `BGP212/MomentAgg.lean`.

Exact demonstrations and scope. Four staged demonstration modules and their assembly file provide kernel-checked rational examples using shared moment tables. The corresponding source files are `Solution/MomentAggDemo1.lean` through `Solution/MomentAggDemo4.lean` and `Solution/MomentAggSuite.lean`. The specification and reproducibility records are `Exploratory/V2j/SPEC.md`, `specs/results/V2j_moment_agg.yaml`, and `specs/checks/moment_agg_v2j_demo.py`. Aggregation is an exact evaluation transformation, not by itself an asymptotic complexity bound, a certificate for every numerical Gram entry, or a certificate for the final variational inequality.

Verification. The `Comparator/moment_agg_v2j.json` configuration covers 24 theorem statements. Its solution was accepted on the Linux VPS and independently on the MacBook Pro, each reporting "Your solution is okay!"; the Mac replay used the non-sandboxing `fake-landrun.sh` development substitute, while the VPS used Linux `landrun`. On the Mac, the V2j core and four individual demonstrations, followed by the combined suite, all compiled successfully. The source revision published to the repository's `main` branch is `a9a8af9`.

14 Symmetric-function reduction of the placement recursion (V2k)

V2i's placement recursion enumerates ways that successive factors of a product of power sums may merge into already selected coordinates or open new coordinates. V2k instead stores canonical states indexed by one integer partition for each of the three coordinate classes. For a partition ν, write $\widetilde{m}_\nu$ for the augmented monomial symmetric function, whose monomials count ordered assignments to distinct coordinates. For a positive integer a, multiplication by a class power sum is represented by

$$p_{c,a}\widetilde{m}_\nu = \sum_{j=1}^{\ell(\nu)} \widetilde{m}_{\nu+ae_j} + \widetilde{m}_{\nu\cup(a)}. \tag{8}$$

The zero-exponent part is handled separately via the factor $n_c - \ell(\nu)$, where n_c is the class population. Canonical states are combined using exact rational coefficients and balanced normalization.

Theorem 14.1 (Equivalence of symmetric and placement evaluation)**.** *For every ordered field, arbitrary moment oracle M, class sizes n, and list of class-dependent polynomial factors φ_s, the symmetric function evaluator equals V2i's original placement recursion:*

$$\mathrm{symPlaceVal}(M, n, \varphi_s) = \mathrm{placeVal}(M, n, 0, \mathrm{Fin.elim0}, \mathrm{Fin.elim0}, \varphi_s).$$

Consequently the V2k piece and entry expressions agree with V2i's, and the corresponding marginal Gram-entry identity follows under the hypotheses of the V2i theorem.

Lean: `symPlaceVal_eq`, `genPieceSym_eq`, `genEntrySym_eq`, `jT_eq_genEntrySym`; `Solution/SymReduce.lean`, `BGP212/SymReduce.lean`. The proof establishes the equivalence in general, rather than only for sample polynomials.

Transition coefficients and Dirichlet moments. For products of positive-degree class power sums, the transition coefficients $R_{\alpha,\nu}$ are independent of the class populations. After integrating the augmented states, each component contributes a falling-factorial count of distinct coordinate assignments and an exact moment of the rough and smooth aggregate degrees:

$$\int \prod_c p_{c,\alpha_c} = \sum_\nu R_{\alpha,\nu} \left(\prod_c (n_c)_{\ell(\nu_c)} \right) \left(\prod_{(c,e)\in\nu} e! \right) M(|\nu_0|, |\nu_1| + |\nu_2|) \,.$$

Here M denotes the specified piece-moment oracle, and the displayed identity abbreviates its Dirichlet-weighted polynomial evaluation. Theorems `psTrans_indep`, `placeVal_psTrans`, and `symPlaceVal_eq_trans` formalize the coefficient and moment bridges. Theorems `genPieceWith_eq_table`, `aggTable_eq` and `genEntryVal_eq_table` give exact coefficient tables for repeated moment keys using a single symmetric evaluation per piece.

Demonstrations and resource scope. Three demonstration files and a combined suite certify small transition examples, degrees four and six using both evaluators, and degree eight and ten using the symmetric evaluator. At Table 3, piece $(r, k) = (6, 3)$, the table for $J_T[P_{(2,2)}, P_{(2,2)}]$ contains 85 moment coefficients; the complete piece is thereby expressed as an exact combination of those moments. This is a *coefficient certificate*, not a complete numerical evaluation: 64 of the 85 moments remain to be evaluated, and the other pieces of that Gram entry remain outstanding. Previously certified V2i and V2j values are recovered in kernel-checked examples. In the reported development-container benchmarks, degree-eight evaluation decreased from 45.1 seconds to 4.98 seconds; these timings are observations, not a complexity theorem.

Independent verification. The 36-statement Comparator configuration `Comparator/sym_reduce_v2k.json` was accepted on the MacBook Pro and on the Linux VPS with native `landrun`. The central V2k proof module and combined demonstration suite built on the Mac. The VPS also compiled the central theorem module; its separate degree-ten demonstration build exited with code 137 on a machine with 3.8 GiB of RAM and no swap, while the full Comparator suite succeeded. The reported theorem axioms are `propext`, `Quot.sound`, and `Classical.choice`. **Provenance:** `a2cb149` (proof), `2b351cd` (verification record); `specs/results/V2k_sym_reduce.yaml`, `Exploratory/V2k/SPEC.md`.

15 Staged symmetric-function certificates and reusable profiles (V2ℓ)

V2k's canonical evaluator reduces placement enumeration, but a single kernel evaluation of high-degree runs can exhaust memory. V2ℓ decomposes the symmetric run into separately checked transitions and merges. Let $L_0 = [(\varnothing, 1)]$ and let $\varphi_1, \ldots, \varphi_s$ be class-dependent polynomial factors. A certified step records that the normalized transition from L_{i-1} yields L_i; for larger states it may instead certify individual chunks followed by a checked merge.

Theorem 15.1 (Composition of staged placement certificates)**.** *For any ordered field, class sizes n, factors φ_s, moment oracle M, and final state L, a certificate* $\mathrm{ChainCert}(n, \varphi_s, [(\varnothing, 1)], L)$ *implies*

$$\mathrm{placeVal}(M, n, 0, \mathrm{Fin.elim0}, \mathrm{Fin.elim0}, \varphi_s) = \mathrm{symFinal}(M, L).$$

At the list level, StepsOK *certifies that successive normalized transitions compose to the corresponding* symRun*.*

Lean: `chainCert_placeVal`, `symRun_of_steps`, `placeVal_of_steps`, `stepCert_of_beq`, `stepCert_runs`, `chainCert_append`, `chainCert_snoc`; `Solution/SymStaged.lean`. The certificates permit prefix reuse and splitting a large factor step into independently checked chunks and a final merge.

Theorem 15.2 (Permutation invariance and profile tables)**.** *Permutation of the factor list preserves V2i's placement value (`placeVal_perm`), as proved from commuting factor operators (`mOp_comm`). Certified staged runs or their moment profiles can be reused to form exact coefficient tables. Theorems `stagedTable_eq`, `profTable_eq`, and `genPieceWith_eq_profTable` identify these table expressions with the corresponding unaggregated piece values.*

Lean: `Solution/SymPerm.lean`, `Solution/SymStaged.lean`, `BGP212/SymStaged.lean`. A profile groups $\sum \kappa \prod e!$ by the two aggregate degrees (A, B); the actual geometric moment oracle remains a separate input.

Degree-twelve demonstrations. Staged files certify six shifted $\widehat{P}_2$ factors on synthetic data and on Table 3 piece $(r, k) = (6, 3)$, with class sizes $(6, 3, 35)$. Previously checked V2i and V2k evaluations are reproduced exactly by Lean reduction. For the single piece of $J_T[P_{(2,2,2)}, P_{(2,2,2)}]$, a certified profile table gives an explicit linear combination of **231 moment keys**. The coefficients are checked, whereas the remaining geometric moment values and the other pieces of this entry still require certificates. The reported development build compiled 61 sequential modules, with each module below 4 GB peak resident memory; the largest measured 3.95 GB in that environment. The exploratory B_{21} enumeration gives 2,361 factor multisets and a 2,360-step sorted prefix trie per piece, not a certified complete trie.

Independent verification and provenance. The central proofs and sequential degree-twelve certificates compiled on the MacBook Pro and the Linux VPS. The **35-statement** Comparator suite was accepted on the Linux VPS with real `landrun` against `8033c48`; the independent Linux verification record is `7ff03bf`. Mac Comparator acceptance has not been established from the saved output. The reported axioms are `propext`, `Quot.sound` and `Classical.choice`, with no `sorry`. Verification artifacts: `Comparator/sym_staged_v2l.json`, `specs/results/V2l_sym_staged.yaml`, and `Exploratory/V2l/SPEC.md`.

16 Certified prefix-trie evaluation and coefficient tables (V2m)

V2ℓ permits certified prefixes to be reused, while the sorted factor-multiset trie from the exploratory enumeration provides a natural organization for such reuse. V2m checks a specified subtree of this construction rather than assuming correctness of the entire exploratory trie. The underlying evaluation agrees with V2i's coordinate-placement recursion.

Theorem 16.1 (Certified trie-node evaluation)**.** *For a certified trie node, the associated symmetric state evaluates to the V2i placement value for its factor path and the specified class sizes, for any moment oracle. In particular, certified prefix extensions preserve the value of the corresponding placement computation; factor-order invariance allows a multiset of factors to share a canonical path.*

Lean: `trieOK_placeVal`; `Solution/SymTrie.lean`, `BGP212/SymTrie.lean`, `Solution/SymTrieNodes.lean`. The proof composes V2ℓ's certified transitions with the V2k/V2i evaluation equivalence. Each recorded profile-only leaf is computed from its parent's full canonical symmetric state before the state is reduced to an aggregate-degree profile. Thus a profile is an output of a certified state calculation, not a generally sufficient state for subsequent transitions.

Certified coverage. The V2m certificate covers **77 nonroot trie nodes** and **78 profiles**, including the root. Of the nonroot nodes, 22 retain complete symmetric states and 55 are profile-only leaves. The selected calculations include six factors of degree two (total degree 12), degree-twenty factors and their product (total degree 40), and reproduction of the earlier degree-twelve profile table by Lean reduction. The Table 3 examples use the single cardinality piece $(r, k) = (6, 3)$, whose three coordinate-class sizes are $(6, 3, 35)$.

Exact coefficient tables. The demonstrated certificates provide, among other examples, a 231-key moment profile for a six-factor degree-twelve block, a 901-key profile for selected pairs of even power sums, and a 523-key coefficient profile for a P_{20}-by-P_{20} piece. These are exact rational coefficients of *unevaluated* geometric moments; they are not claims of numerical values for the corresponding complete marginal Gram entries. In the larger Table 3 calculation, the certificate system checks 79,553 stored coefficient entries, with chunked proofs to limit per-module resource requirements.

Independent verification and provenance. The central V2m proofs and staged certificate suite compiled on macOS and on the Linux VPS, the latter reporting "Build completed successfully (3103 jobs)." The **20-statement** Comparator configuration `Comparator/sym_trie_v2m.json` returned "Lean default kernel accepts the solution" and "Your solution is okay!" on the VPS with real Linux `landrun`. The independently tested source commit is `73b5359`; it was merged into `main` by `283b634`. The verification record is `specs/results/V2m_sym_trie.yaml`; Mac Comparator acceptance is not independently confirmed. Build instructions, controls, and the reproducible exploratory generator are in `specs/checks/sym_trie_v2m_build.sh`, `specs/checks/sym_trie_v2m_controls.py`, and `specs/checks/sym_trie_v2m_demo.py`. The remaining trie nodes, other Table 3 cardinality pieces, full 846×846 numerical matrices, optimized polynomial $P^\star$, and the final variational positivity certificate remain outstanding.

17 Verification, limitations, and further mathematics

The project uses Lean `v4.34.0-rc2` and a pinned Mathlib revision `85e3a25`. At the V2g baseline, all 19 Comparator suites were independently accepted on the VPS. Four positive V2d controls were accepted; four intentionally incorrect controls were rejected with unsolved goals. The V2d proof commit is `52fd36a`; its published verification record is commit `90055db`. For V2e-A, five positive controls were accepted and four negative controls were rejected. The V2e-A proof commit is `01644e6`, with VPS verification/provenance commit `38606c0`. For V2e-B, five positive controls were accepted and five deliberately incorrect controls rejected; the V2e-B proof commit is `6a21780`, with verification/provenance commit `25a7376`. For V2f, four positive Lean-file controls were accepted and five negative controls rejected. Its proof commit is `8c50fb6`, with VPS provenance commit `da0c3fd`. For V2g, two positive controls were accepted and four negative controls rejected; its 266 certificate lemmas regenerated byte-identically on the VPS. The V2g proof commit is `ad70abb`, with verification/provenance commit `12f8b16`. For V2h, the 27-statement Comparator accepted the power-sum-square certificate on the VPS. Its proof commit is `6cef1f4`, with VPS provenance commit `a9b0e0c`. The recorded V2h proof includes 14 rational moment-certificate rows and an assembly theorem. The final Comparator acceptance is confirmed; a complete 20/20 regression count has not been established from a saved log. For V2i, the 33-statement Comparator suite was accepted by the Lean default kernel on the VPS. The complete regression log records **21/21** "Your solution is okay!" messages and the verification script's **exit status 0**. The proof snapshot is `a994d73`; its separately published VPS provenance commit is `486cf85`. The V2i record and Comparator configuration are `specs/results/V2i_gen_entry.yaml` and `Comparator/gen_entry_v2i.json`. Prior control results and exploratory numerical comparisons are distinct from the saved VPS kernel acceptances. For V2j, the 24-statement Comparator

configuration was accepted on both the VPS and the MacBook Pro against source commit a9a8af9. The Mac's Comparator setup used a Lean 4.34.0-rc2-compatible `lean4export`; its non-sandboxing development substitute is distinct from Linux landrun isolation. These V2j acceptances are individual suite checks, not a newly asserted complete 22-suite regression. For V2k, the 36-statement Comparator suite passed independently on both the MacBook Pro and the Linux VPS, against source commit a2cb149; provenance is published at 2b351cd. The Linux VPS exceeded its memory budget on the standalone degree-ten demonstration build, but accepted the full Comparator suite. For V2ℓ, the staged certificate modules built independently on Mac and VPS at source commit 8033c48; the 35-statement Comparator passed on Linux with real `landrun`. Verification provenance is commit 7ff03bf. For V2m, both macOS and Linux sequential certificate builds succeeded, and the Linux VPS accepted the 20-statement Comparator using native `landrun`; see the source commit 73b5359, merge commit 283b634, and `specs/results/V2m_sym_trie.yaml`. The reported kernel axioms for these developments are `propext`, `Quot.sound`, and `Classical.choice`. Results and verification scripts are available in specs/results.

Further work evaluates additional entries using V2i's general representation, V2j's exact aggregation, V2k's symmetric-state coefficient tables, V2ℓ's staged moment profiles, and V2m's certified trie tables; it also improves the shared-moment and cell computations, and addresses optimized polynomial coefficients, exact variational positivity certificates, and remaining analytic hypotheses. The existing Lean development formalizes the *conditional* implication $\mathrm{DHL}[45, 2] \Rightarrow H_1 \leq 212$; this manuscript does not claim an unconditional formal proof of $H_1 \leq 212$.

Acknowledgment and provenance

The bounded-prime-gap application is motivated by Axiom Math's mathematical argument [1]. This manuscript records an independent Lean formalization of the specified results, with theorem names and commit identifiers to support reproducible review. Substantive exposition and attribution will be expanded in a subsequent draft.

References

[1] Axiom Math, *A New Bound for Small Gaps Between Primes*, preliminary draft, 3 September 2026. Project page: https://primegaps.axiommath.ai/.

[2] Reading Point, *lean-bgp212*: Lean source, verification records, and certificates. https://github.com/thinkthoughts/lean-bgp212.